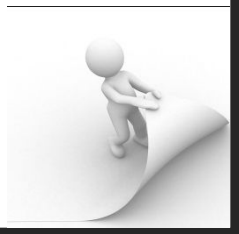
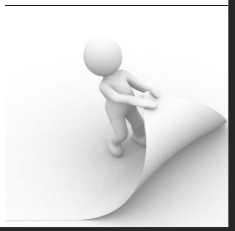
C++11新特性之三：基于范围的for循环与自动类型推导

张锦成 2015/10/14



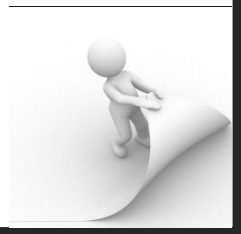
Outline

- 右尖括号>的改进
- 基于范围的for循环
- auto类型推导
- decltype类型推导



1

右尖括号>的改进



右尖括号>

类模板实例化

```
vector<vector<int>> vec;
vector<vector<int> > vec;
```

类型转换

```
const vector<int> vec = static_cast<vector<int>>(v);
const vector<int> vec = static_cast<vector<int> >(v);
```

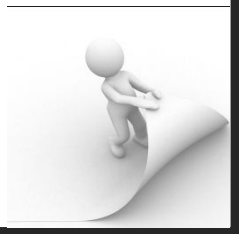
右尖括号>

C++11 允许两个连续的右尖括号>进行实例化或类型转换

C++98 则不允许

2

基于范围的for循环

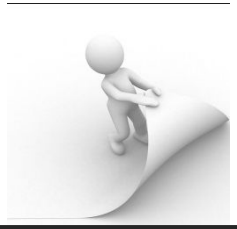


for loop

C++98

循环体需要指定循环的界限范围

```
int a[]={1, 2, 3, 4, 5};
for(int i=0; i<5; i++) {
    std::cout << a[i] << std::endl;
}

int print(int i) {
    std::cout << i << std::endl;
}
std::for_each(a, a+sizeof(a)/sizeof(a[0]), print);
```



for loop

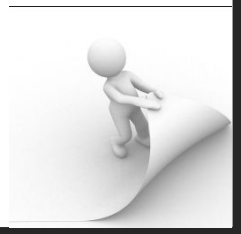
- C++11

自动范围界定

```
int a[]={1, 2, 3, 4, 5};
for(int i:a) {
    std::cout << i << std::endl;
}
```

3

auto类型推导



类型推导

- 静态类型语言

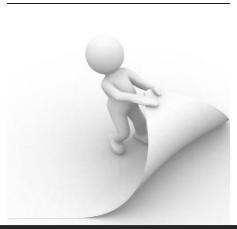
  编译阶段进行类型检查

  C/C++, Java, etc.

- 动态类型语言

  运行阶段进行类型检查

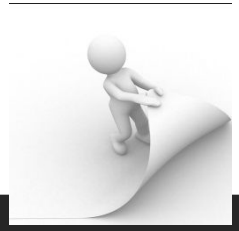
  Python, JavaScript, Lua, etc.



python案例

```
name = "hello world!"
print name
```

- 定义变量的时候不需要申明变量的类型
- 程序运行的时候依赖于类型推导来推导出变量类型

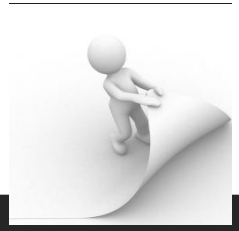


C++案例

```
string name = "hello world";
std::cout << name << std::endl;
```

- ✓ 定义变量的时候必须申明变量的类型
- ✓ 编译阶段检查变量类型

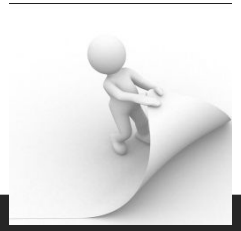
C++可以像Python那样不申明变量类型么？



auto类型推导

```
double func(){
    return 1.0;
}

class MyClass{
    int i;
}sample;


int main()
{
    auto i = 1;
    auto j = func();
    auto k = sample;

    cout << typeid(i).name() << endl;
    cout << typeid(j).name() << endl;
    cout << typeid(k).name() << endl;
    return 0;
}
```

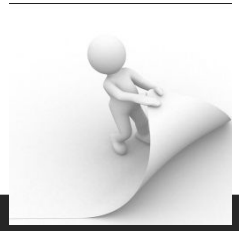
编译器在编译时期将auto替代为变量的实际类型



auto的优势

- 复杂类型变量申明时简化代码

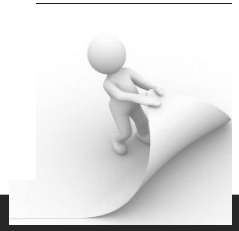
```
std::map<int, std::string> m = {{1, "a"}, {2, "b"}};

for(std::map<int, std::string>::iterator iter = m.begin(); iter!=m.end(); ++iter){
    std::cout << iter->first << " " << iter->second << std::endl;
}


for(auto iter = m.begin(); iter!=m.end(); iter++) {
    std::cout << iter->first << " " << iter->second << std::endl;
}


for(auto iter:m) {
    std::cout << iter.first << " " << iter.second << std::endl;
}
```



auto的优势

免除程序员在一些不确定类型的变量申明时的麻烦或者错误

```
class PI{
    public:
        double operator* (float v) {
            return (double)val * v; // extend the precision
        }
        const float val = 3.1415927f;
};

int main()
{
    float radius = 1.0;
    PI pi;
    auto area = 2 * (pi * radius);

    std::cout << area << std::endl;

    return 0;
}
```



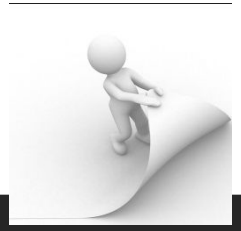
auto的优势

auto也能在某些情况下带来程序性能的提升

```
#define MAX1(a, b) ((a)>(b))?(a):(b)

#define MAX2(a, b)({\
        auto _a = (a);\
        auto _b = (b);\
        (_a>_b) ? _a : _b;})

int m1 = MAX1(1*2, 1+2);
int m2 = MAX2(1*2, 1+2);
```

第二种方法更高效，只计算1次表达式的值

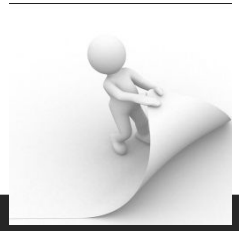
如果参数带有自增或自减操作符，第一种就会得到错误的结果



auto使用细则

- auto定义变量的同时要初始化

```
auto x = 1;

auto z; // error here, cannot deduce the type of z
z = x;
```

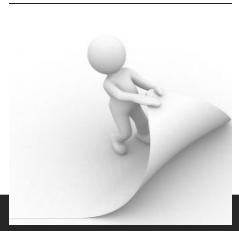


auto使用细则

auto类型指示符与指针

```
class Test{
    int i;
};

int main()
{
    int x;

    auto a = &x;
    auto * b = &x;

    std::cout << typeid(a).name() << std::endl;
    std::cout << typeid(b).name() << std::endl;

    auto test = new Test();
    std::cout << typeid(test).name() << std::endl;
    return 0;
}
```

auto用于指针，*可有可无

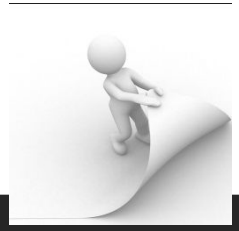


auto使用细则

auto类型指示符与引用

```
int x;
auto &a = x;

std::cout << typeid(a).name() << std::endl;
```

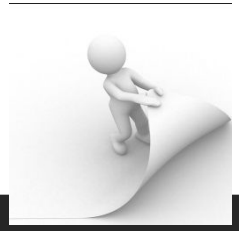
申明一个变量是另一个变量的引用，必须使用auto &



auto使用细则

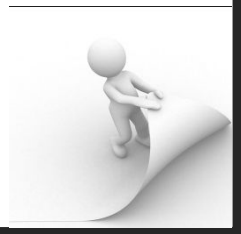
auto不适用的某些场景

```
void fun(auto x = 1) {} // error here, auto函数参数

struct str{
    auto i = 1;   // error here, auto非静态成员变量
};

int main()
{
    char x[3];
    auto y = x; // char*, a pointer that points to char
    std::cout << typeid(y).name() << std::endl;  // Pc

    auto z[3] = x; // error here, auto数组

    std::vector <auto> v = {1}; // error here, auto模板参数

    return 0;
}
```



4

decltype类型推导



decltype

1. decltype类型推导同auto类型推导一样都是编译时期进行
2. decltype只对表达式进行类型推导
3. 提升了C++11泛型编程能力

```
int i;
decltype(i) j;

double d;
decltype(i+d) m;

cout << typeid(j).name() << " " << typeid(m).name() << endl;
```

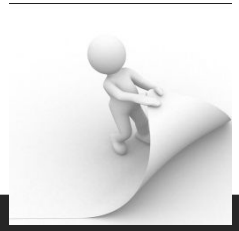
decltype

```
template<typename T, typename U>
void sum(T t, U u) {
    auto val = t + u;
    cout << val << endl;
    // do some stuff with val
}

template<typename T, typename U>
auto sum2(T t, U u) -> decltype(t+u){
    auto val = t + u;

    // do some stuff with val

    return val;
}
```

References

1. 《深入理解C++11: C++11新特性解析与应用》第4章

2. C++11新特性: http://www.cocos.com/doc/tutorial/show?id=1120



THANK
YOU!